JSPM's Bhivarabai Sawant Institute of Technology and Research

Wagholi, Pune – 412207

Department of Computer Engineering

MINI PROJECT REPORT

Dynamic COVID-19 Information Website with Software Testing and Quality Assurance

(Software Testing & Quality Assurance)

Laboratory Practice–IV

Submitted To:

Dr. Phadtare Tushar

Submitted By:

Shivshankar Mali

Division: C

Roll No.: 49

Academic Year: 2026–27

Semester: VII

Abstract

Digital images are frequently used as evidence in cybercrime investigations, but image files may be edited or re-saved after they are created. This mini project presents an Image Forensics Tool developed in Python with a graphical user interface using Tkinter. The tool displays basic image information, calculates a SHA-256 hash to support file-integrity checking, and generates an Error Level Analysis (ELA) preview. The report area displays information such as the file name, path, file size, image format, dimensions, colour mode, hash value, and available EXIF metadata. The project is intended as an educational aid for inspecting images. ELA and metadata findings are indicators for further examination, not proof that an image has been manipulated.

Contents

Abstract i

1 Introduction 1
- 1.1 Problem Statement . . . 1
- 1.2 Objectives . . . 1
- 1.3 Scope . . . 2

2 Technology Used 3

3 System Design 4
- 3.1 Main Modules . . . 4
- 3.2 System Flow . . . 5

4 Implementation 6
- 4.1 Project Components . . . 6
- 4.2 SHA-256 Integrity Check . . . 6
- 4.3 Error Level Analysis . . . 6
- 4.4 Metadata Extraction . . . 7

5 Output and Screenshots 8
- 5.1 Main Application and Forensic Report . . . 8
- 5.2 Application Interface . . . 8
- 5.3 ELA Result . . . 9

6 Testing and Results 11

7 Limitations 12

8 Conclusion 13
- 8.1 Future Scope . . . 13

References 14

List of Figures

3.1 Workflow of the Image Forensics Tool 5

5.1 Image Forensics Tool with selected image and report 8
5.2 Image preview and Error Level Analysis result 9
5.3 ELA output for the sample image 10

Chapter 1

Introduction

Digital image files can be important in investigations involving online fraud, misinformation, identity misuse, and other cyber incidents. An investigator may need to inspect image properties and look for visual inconsistencies. This project provides a simple desktop application that combines image preview, metadata reporting, SHA-256 hashing, and Error Level Analysis in one interface.

1.1 Problem Statement

To design and develop a desktop tool that assists in the preliminary forensic examination of digital images by displaying file properties, calculating a SHA-256 hash, reading available EXIF metadata, and producing an Error Level Analysis preview.

1.2 Objectives

1. Build a user-friendly image forensics interface using Python and Tkinter.
2. Allow users to select and preview an image file.
3. Display file name, path, size, format, dimensions, and colour mode.
4. Calculate a SHA-256 hash to support integrity comparison.
5. Read available EXIF metadata from the selected image.
6. Generate and save an ELA image for preliminary visual inspection.
7. Present results in a readable forensic report area.

1.3 Scope

The application is a preliminary analysis tool for common image formats supported by the installed imaging library. It does not automatically determine whether an image is authentic or forged. Results should be interpreted alongside original files, source information, and other forensic methods.

Chapter 2

Technology Used

Technology	Purpose
Python	Implements file handling, hashing, metadata extraction, and ELA processing.
Tkinter	Provides the desktop graphical user interface.
Pillow (PIL)	Loads images, resizes previews, reads EXIF data, and creates ELA output.
hashlib	Calculates the SHA-256 digest of the selected image file.
OS / pathlib	Reads file names, paths, and file sizes.
VS Code	Used to write and run the program.
LaTeX / Overleaf	Used to prepare this project report.

Table 2.1: Technologies used in the project

Chapter 3

System Design

3.1 Main Modules

- **Image selection:** Opens an image from the user's computer.
- **Image preview:** Displays the selected image in the application.
- **Metadata extraction:** Reports file properties and any available EXIF metadata.
- **SHA-256 integrity:** Calculates a hash that can be compared with a hash from another examination.
- **Error Level Analysis:** Re-saves the image at a selected JPEG quality and visualizes pixel differences.
- **Save ELA image:** Allows the generated ELA result to be saved.
- **Forensic report:** Displays the collected properties and analysis notes.

3.2 System Flow

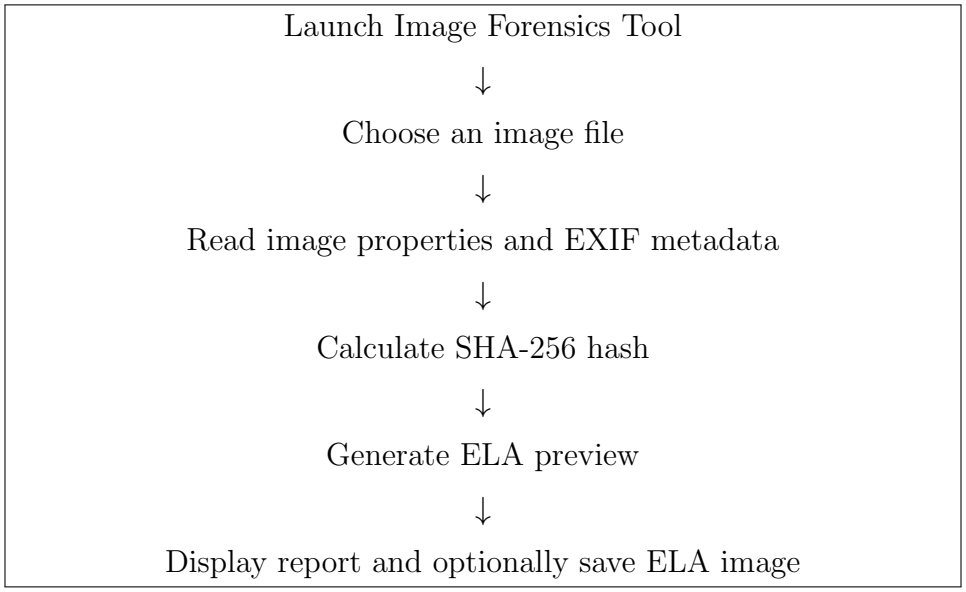


Figure 3.1: Workflow of the Image Forensics Tool

Chapter 4

Implementation

4.1 Project Components

The application is implemented as a Python desktop program. Tkinter provides buttons and preview panels, while Pillow performs image operations. Python's `hashlib` module is used for SHA-256 calculation. The main interface includes controls for choosing an image, saving the ELA result, and clearing the current analysis.

4.2 SHA-256 Integrity Check

The application reads the image file as bytes and calculates its SHA-256 digest. If the file changes, its hash will normally change as well. A hash identifies the exact byte content of a file; it does not by itself prove who created the image or whether the depicted scene is genuine.

Listing 4.1: Example SHA-256 calculation

```
import hashlib

def calculate_sha256(file_path):
    sha256 = hashlib.sha256()
    with open(file_path, "rb") as image_file:
        for block in iter(lambda: image_file.read(4096), b""):
            sha256.update(block)
    return sha256.hexdigest()
```

4.3 Error Level Analysis

Error Level Analysis (ELA) compares an image with a JPEG re-encoding of that image. Differences are amplified to make compression inconsistencies easier to inspect. These

differences may arise from editing, but also from normal image content, image format, or previous compression. Therefore, ELA should be treated as a clue rather than a definitive manipulation detector.

4.4 Metadata Extraction

The tool displays general file properties, including the file name, full path, file size, image format, pixel dimensions, and colour mode. It also attempts to read EXIF metadata when present. Many images, especially screenshots or images shared through online services, may not contain EXIF information.

Chapter 5

Output and Screenshots

5.1 Main Application and Forensic Report

Figure 5.1 shows the Image Forensics Tool with a selected test image, ELA preview, and the forensic report area displaying file properties and the SHA-256 digest.

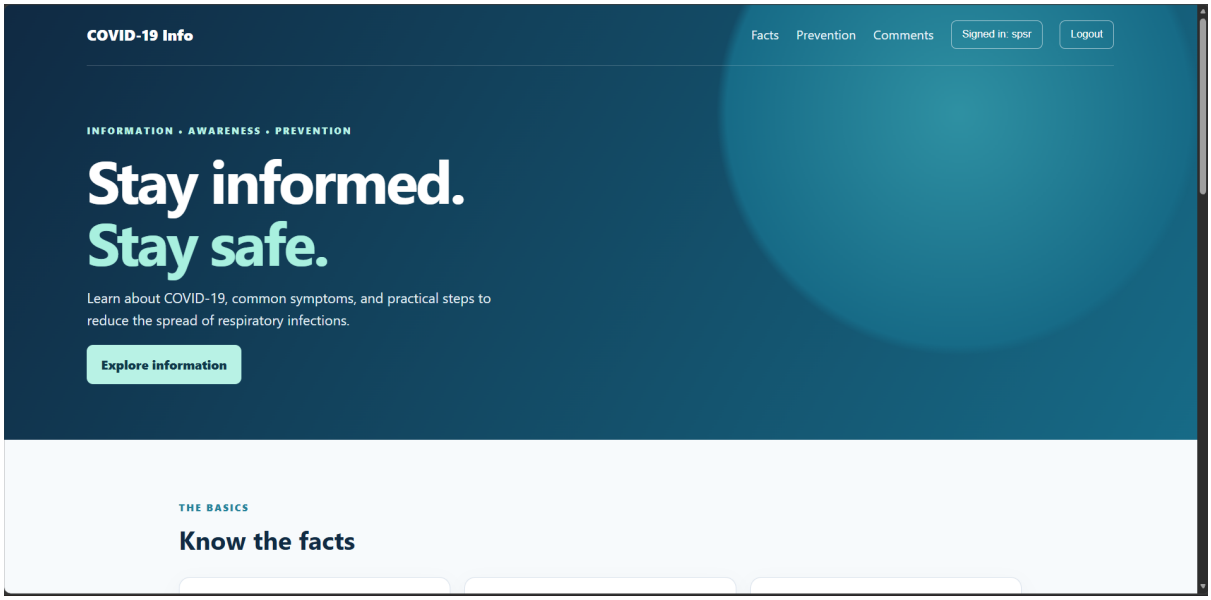


Figure 5.1: Image Forensics Tool with selected image and report

5.2 Application Interface

Figure 5.2 shows another view of the application interface and the generated ELA preview.

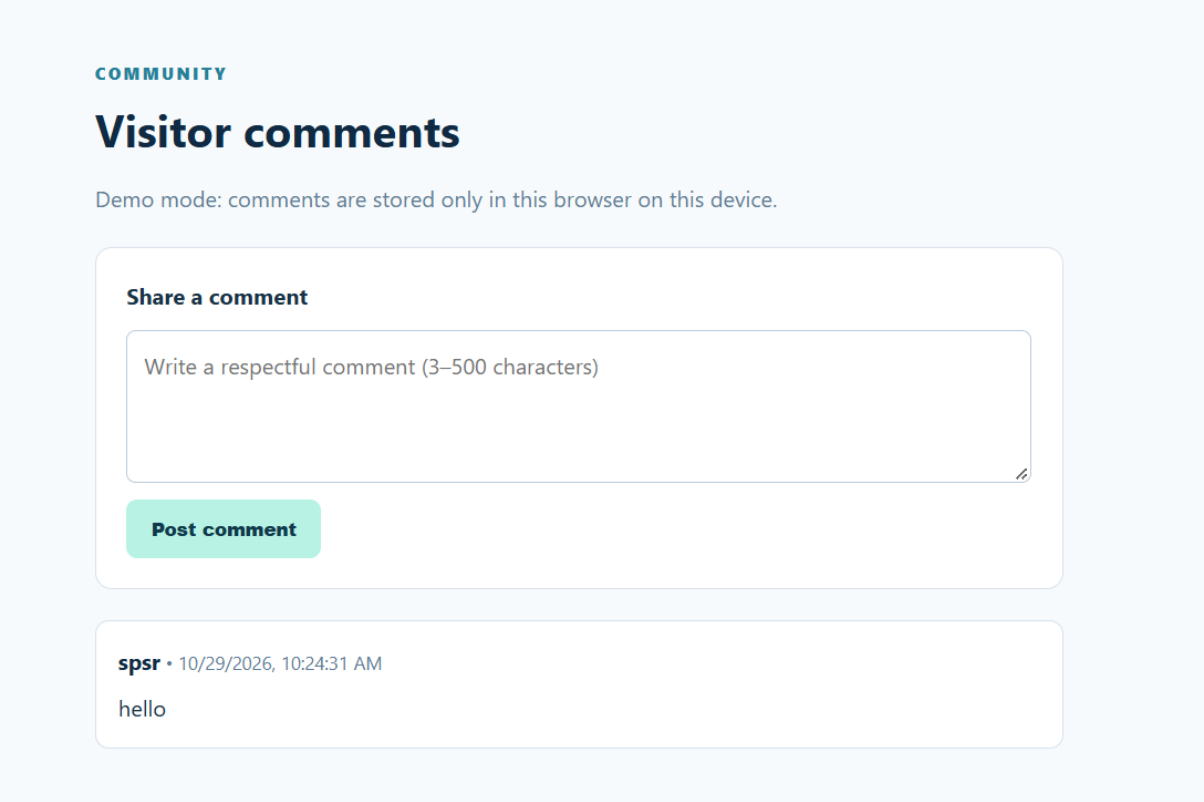


Figure 5.2: Image preview and Error Level Analysis result

5.3 ELA Result

Figure 5.3 shows the ELA output generated for the selected sample image. Bright or coloured areas indicate differences after re-encoding; they should be examined carefully and not automatically interpreted as proof of tampering.

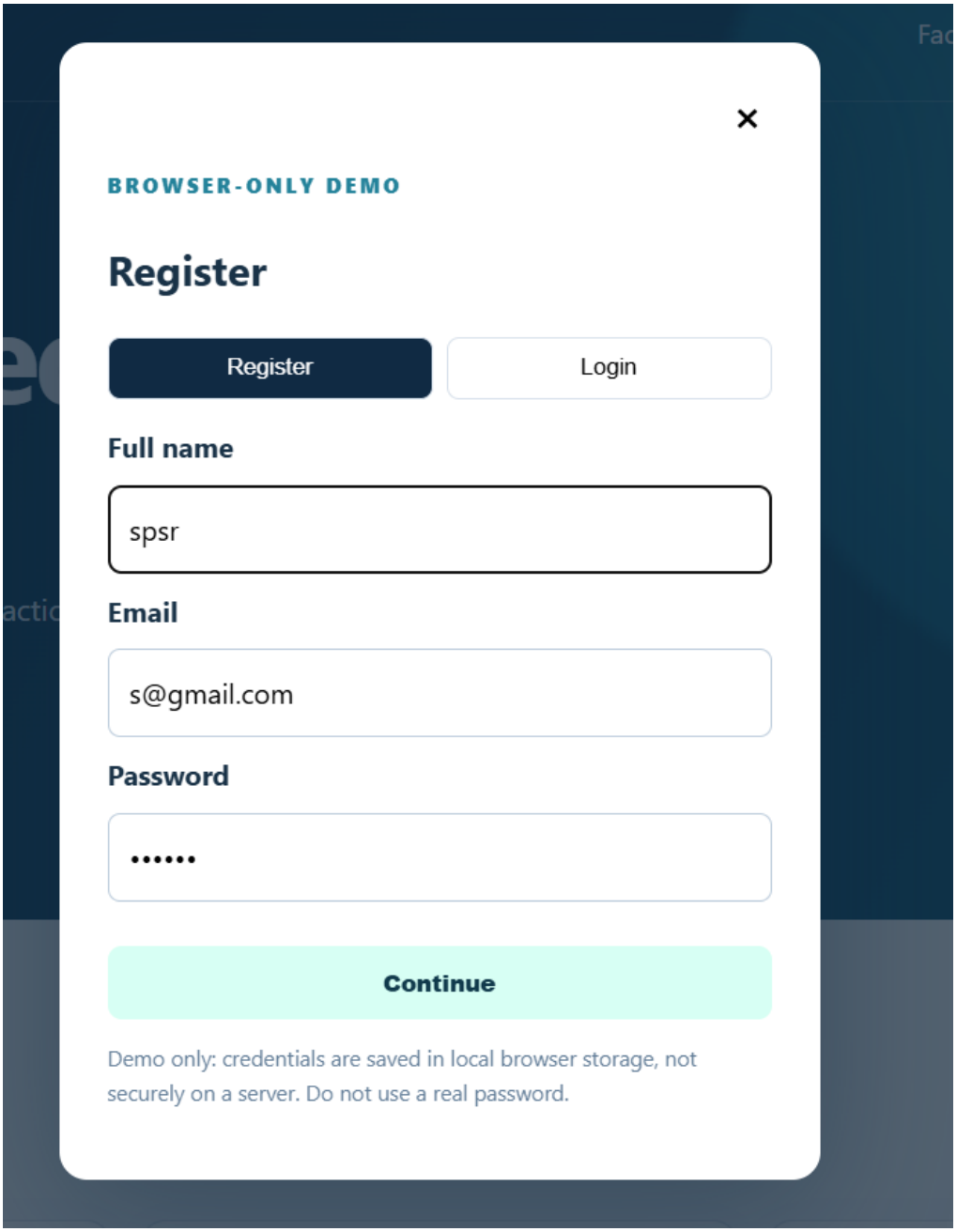


Figure 5.3: ELA output for the sample image

Chapter 6

Testing and Results

Test case	Expected result	Status
Launch application	Main window and controls appear	Pass
Select supported image	Image preview is displayed	Pass
Inspect file properties	Name, size, format, and dimensions are shown	Pass
Calculate SHA-256	Hash digest is displayed in report	Pass
Image contains EXIF	Available metadata is listed	Conditional
Image has no EXIF	Application reports no EXIF metadata found	Pass
Generate ELA	ELA preview appears in the right panel	Pass
Save ELA output	User can save the generated ELA image	Pass
Clear analysis	Preview and report return to initial state	Pass

Table 6.1: Functional test cases for the Image Forensics Tool

Note: The statuses above describe the intended observed behaviour of the demonstrated application. Repeat each test on the final program before submitting the report.

Chapter 7

Limitations

- ELA is a preliminary visual technique and cannot conclusively prove image manipulation.
- EXIF metadata may be absent, stripped, or modified.
- SHA-256 supports integrity comparison but does not establish image authenticity by itself.
- The tool's results depend on the image format and the processing parameters used.
- A full forensic investigation may require additional tools, chain-of-custody procedures, and corroborating evidence.

Chapter 8

Conclusion

The Image Forensics Tool provides a simple desktop interface for preliminary digital image examination. It combines image preview, file-property reporting, EXIF metadata inspection, SHA-256 hashing, and Error Level Analysis. The project demonstrates how Python libraries can support basic forensic workflows and how results can be documented in a structured report. The tool should be used as an aid to investigation rather than as an automatic verdict on authenticity.

8.1 Future Scope

- Add batch processing for multiple images.
- Export the forensic report as PDF or text.
- Add metadata comparison between original and processed images.
- Record analysis timestamps and tool settings for reproducibility.
- Add additional forensic indicators while clearly describing their limitations.

References

1. Python Documentation, https://docs.python.org/3/
2. Pillow Documentation, https://pillow.readthedocs.io/
3. Python `hashlib` Documentation, https://docs.python.org/3/library/hashlib.html
4. Python Tkinter Documentation, https://docs.python.org/3/library/tkinter.html
5. Krawetz, N. (2007), A Picture's Worth: Digital Image Analysis and Forensics, online technical article.